

Assignment 2 - Molecular dynamics model of water

Rasserismus (Rasmus), Bundamin (Benjamin), Asgerholiker (Asger)

January 2026

1 Create a Struct-of-Arrays version of the program

Code is listed in the last section, and the video in the submissions page.

2 Investigate the performance of the code with a profiler

For each run we wanted to get at least 1 second of run time on each simulation, so for $n = 10$ we ran for 1,000,000 steps, $n = 100$ for 100,000 steps and $n = 1,000$ and $n = 40,000$ for 10,000 steps.

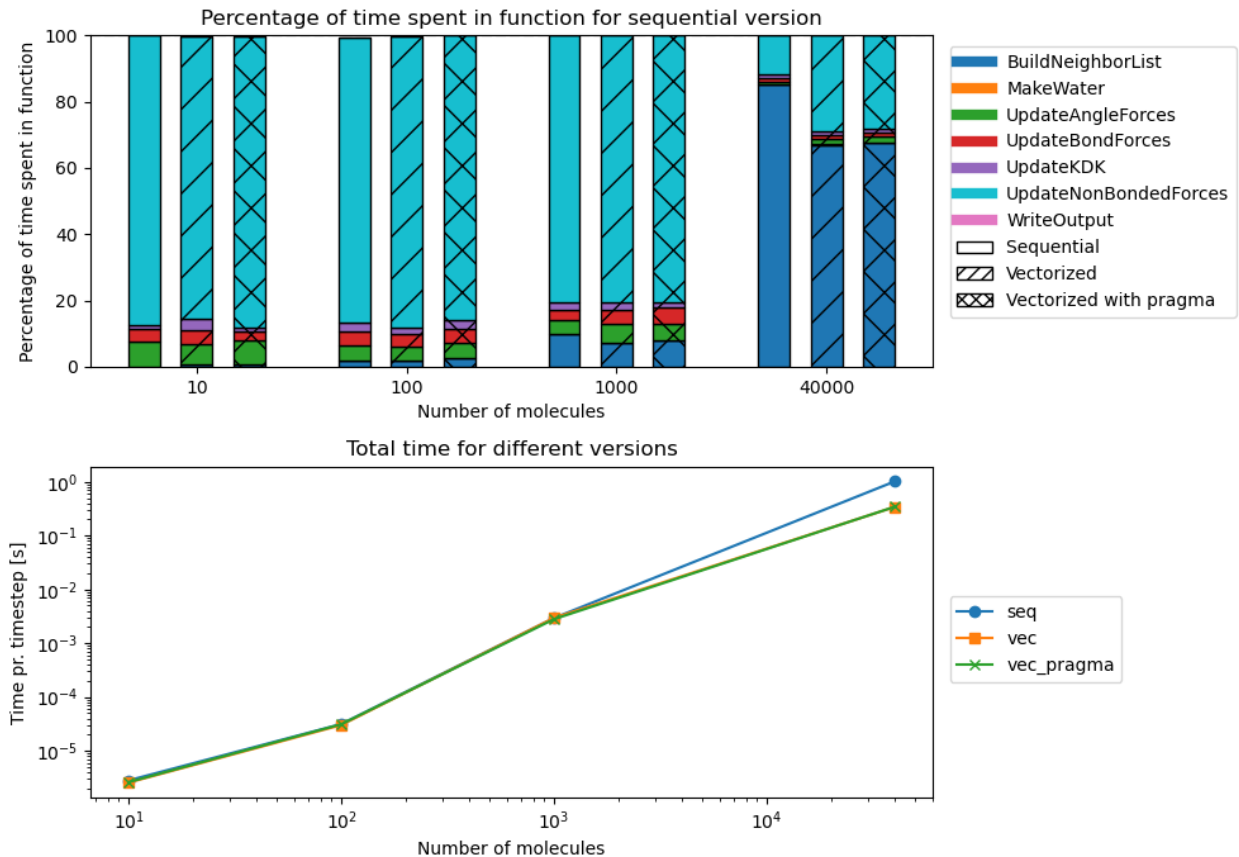


Figure 1: **Bottom:** Here we see a bar plot showing the percentage of time spend in each of the function. For $n \leq 1,000$ *UpdateNonBondedForces* takes the most time, but for 40,000 it is *BuildNeighborList*. **Bottom:** We see the total time taken for each of the different versions of the code. For all runs the most optimal compiler settings were used (OPT: -O3 -ffast -math-pg).

The algorithms we are mainly looking at are all the ones updating forces and building the neighbor list. The functions updating forces are all approximately $O(n)$ whilst the function building the neighbor list is $O(n^2)$, hence we naturally see the *BuildNeighborList* dominate the compute time for bigger values of n . Furthermore, the speedup gain from vectorizing is not found in *BuildNeighborList* since it is nearly impossible to vectorize due to the non-uniformity between the instances in the loops (the same calculations should be applied for all instances of the loop to do vectorization). It's clear from 1 that *UpdateNonBoundForces* dominate the compute time for lower values of n (10 - 1,000). The reason being that it technically has time complexity $O(n \cdot k)$, where k is the number of neighbors. But since k is set constant it scales the same as the other *Update[something]*¹ functions. To acquire lower computation time we would need to optimize *BuildNeighborList* and *UpdateNonBoundedForces*.

Replacing the AoS architecture with a SoA layout does not yield significant speedup for smaller systems, as seen in the lower part of fig. 1. But at a larger scale of 40,000 molecules, the program runs about 3x faster using the SoA version. Observing the bar chart in fig. 1 once again, we see that this is primarily a result of the efficient use of memory in the *BuildNeighborlist* function: Instead of accessing the position of each oxygen atom individually, we instead loop over the same position array in a linear fashion, allowing optimal use of the cache, by increasing the hit rates of L1, L2 and L3 (In order of increasing complexity). Furthermore, the total time for the different versions indicate that for system sizes ranging from 10 to 1000 molecules the two vectorized version has no computational edge over the purely sequentialized. This indicates that this range is purely memory bound, i.e. nearly all the time is spent on setup and memory movement, while the CPU instantly crunches the computations². When the system reaches 40000 molecules the graph clearly illustrates a drastic increase in compute time, but since the two vectorized versions gain ground in terms of computation speed compared to the purely sequentialized version the system is definitely not purely memory bound.

3 Adding OpenMP SIMD pragmas to the code

We saw no speedup when testing the different pragmas, rather they seemed to sometimes yield slightly slower performance. Instead of benchmarking each pragma separately for only 16 molecules, we applied all the pragmas at once, using the same number of steps and molecules as when comparing the sequential and vectorized version. The results are seen in fig. 1. Pragmas were placed before the for loops in *Update[something]* functions and also in the nested for loop in *BuildNeighbourList*. We choose these for loops as they were the ones on the form *for (size_t i = 0; i < N ; i++)* which are most suitably for vectorization. In the *UpdateBondForces*, *UpdateAnglesForces* and *UpdateNonbondedForces* we also added *reduction(+:accumulated_forces_bond)*. It's clear that since the ratio between *BuildNeighborList* and all the *Update[something]* functions doesn't change when pragmas are applied, we see no further improvement in compute time. This can trivially be seen in the second plot showing the total computation time/no. of steps, where *vec* and *vec_pragma* coincide. Based on this we conclude, that the compiler is simply good enough for our vectorized code that it doesn't need further specifications.

¹Refers to all the functions with update in the name

²The different cache usage is illustrated in the appendix 2

4 Appendix

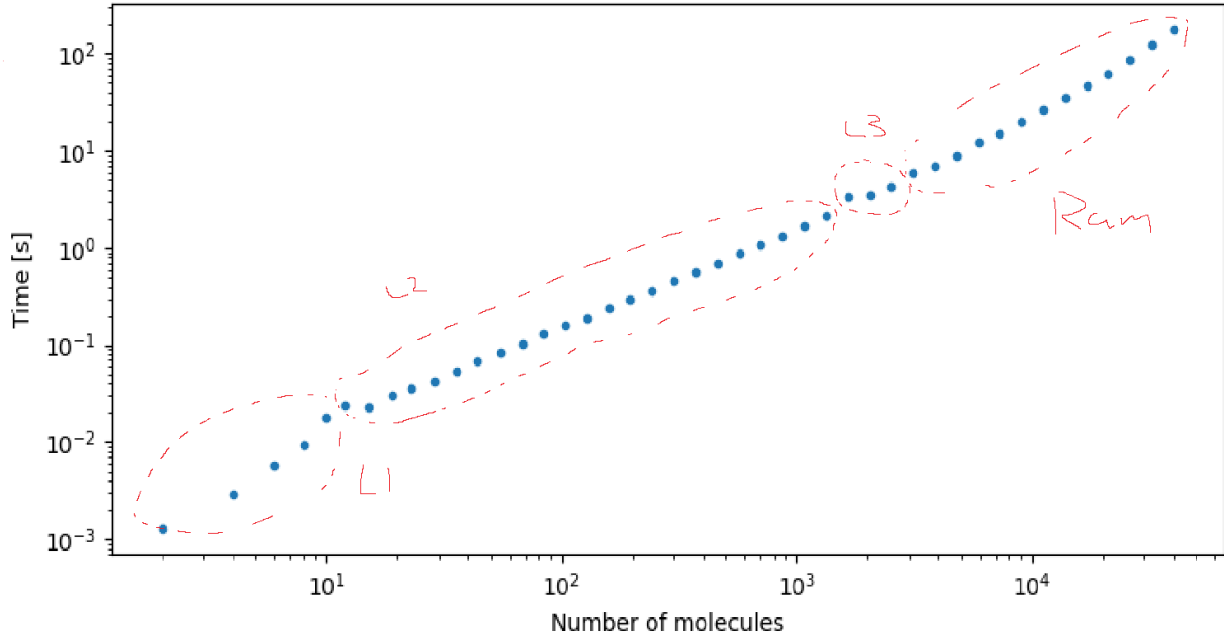


Figure 2: Time plotted against the number of molecules simulated. This is not scientific in any way but the "discontinuity" jumps indicates an addition of a constant time penalty which could be the result of the difference in speed between the different caches and RAM (slower memory transportation time: $t_{L1} < t_{L2} < t_{L3} < t_{RAM}$).

5 Vectorized code

```
#include <iostream>
#include <iomanip>
#include <fstream>
#include <vector>
#include <cassert>
#include <math.h>
#include <chrono>
#include <numeric>
#include <algorithm>

const double deg2rad = acos(-1)/180.0; // pi/180 for changing degs to radians
double accumulated_forces_bond = 0.; // Checksum: accumulated size of forces
double accumulated_forces_angle = 0.; // Checksum: accumulated size of forces
double accumulated_forces_non_bond = 0.; // Checksum: accumulated size of forces
constexpr size_t nClosest = 8; // Number of closest neighbors to consider in n
// constexpr just helps the compiler to optimize the code better

class Vec3 {
public:
    double x, y, z;
    Vec3(double x, double y, double z): x(x), y(y), z(z) {} // initialization of vector
```

```

double mag2() const{                                     // squared size of vector (sl
    return x*x+y*y+z*z;
}
double mag() const{                                     // size of vector
    return sqrt(mag2());
}
Vec3 operator-(const Vec3& other) const{                // subtraction of two vectors
    return {x - other.x, y - other.y, z - other.z};
}
Vec3 operator+(const Vec3& other) const{                // addition of two vectors
    return {x + other.x, y + other.y, z + other.z};
}
Vec3 operator*(double scalar) const{                   // multiplication of vector by
    return {scalar*x, scalar*y, scalar*z};
}
Vec3 operator/(double scalar) const{                   // division of vector by scalar
    return {x/scalar, y/scalar, z/scalar};
}
Vec3& operator+=(const Vec3& other){                    // add and assign to vector
    x += other.x; y += other.y; z += other.z;
    return *this;
}
Vec3& operator-=(const Vec3& other){                    // subtract and assign to vector
    x -= other.x; y -= other.y; z -= other.z;
    return *this;
}
Vec3& operator*=(double scalar){                       // multiply and assign to vector
    x *= scalar; y *= scalar; z *= scalar;
    return *this;
}
Vec3& operator/=(double scalar){                       // divide and assign to vector
    x /= scalar; y /= scalar; z /= scalar;
    return *this;
}
};
Vec3 operator*(double scalar, const Vec3& y){           // multiplication of scalar by ve
    return y*scalar;
}
Vec3 cross(const Vec3& a, const Vec3& b){
    return { a.y*b.z-a.z*b.y,
             a.z*b.x-a.x*b.z,
             a.x*b.y-a.y*b.x };
}
double dot(const Vec3& a, const Vec3& b){
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

/* class for the covalent bond between two atoms  $U=0.5k(r_{12}-L_0)^2$  */
class Bond {
public:
    double K;      // force constant
    double L0;     // relaxed length

```

```

    size_t a1, a2; // the indexes of the atoms at either end
};

/* class for the angle between three atoms  $U=0.5K(\phi_{123}-\phi_0)^2$  */
class Angle {
public:
    double K;
    double Phi0;
    size_t a1, a2, a3; // the indexes of the three atoms, with a2 being the centre atom
};

/* molecule class */

// =====
// Two new classes arranging Atoms in a Structure-of-Array data structure
// =====

/* atoms class, representing N instances of identical atoms */
class Atoms {
public:
    double mass; // The mass of the atom in (U)
    double ep; // epsilon for LJ potential
    double sigma; // Sigma, somehow the size of the atom
    double charge; // charge of the atom (partial charge)
    std::string name; // Name of the atom
    // the position in (nm), velocity (nm/ps) and forces (k_BT/nm) of the atom
    std::vector<Vec3> p,v,f;
    // constructor, takes parameters and allocates p, v and f properly
    Atoms(double mass, double ep, double sigma, double charge, std::string name, size_t N)
    : mass{mass}, ep{ep}, sigma{sigma}, charge{charge}, name{name},
      p{N*identical, {0,0,0}}, v{N*identical, {0,0,0}}, f{N*identical, {0,0,0}}
    {}
};

/* molecule class */
class Molecules {
public:
    std::vector<Atoms> atoms; // list of atoms in the molecule
    std::vector<Bond> bonds; // the bond potentials, eg for water th
    std::vector<Angle> angles; // the angle potentials, for water just
    std::vector<std::vector<size_t>> neighbours; // indices of the neighbours
    size_t no_mol;

    // constructor, takes parameters and allocates neighbour list vector properly
    Molecules(std::vector<Atoms> atoms, std::vector<Bond> bonds, std::vector<Angle> angles,
    : atoms{atoms}, bonds{bonds}, angles{angles}, neighbours{no_mol}, no_mol{no_mol}
    {}
};

// =====

/* system class */
class System {

```

```

public:
    Molecules molecules;           // all the molecules in the system
    double time = 0;               // current simulation time
};

class Sim_Configuration {
public:
    size_t steps = 10000;         // number of steps
    size_t no_mol = 100;          // number of molecules
    double dt = 0.0005;           // integrator time step
    size_t data_period = 100;     // how often to save coordinate to trajectory
    std::string filename = "trajectory.txt"; // name of the output file with trajectory
    // system box size. for this code these values are only used for vmd, but in general

    // simulation configurations: number of step, number of the molecules in the system,
    // IO frequency, time step and file name
    Sim_Configuration(std::vector<std::string> argument){
        for (size_t i = 1; i<argument.size() ; i += 2){
            std::string arg = argument.at(i);
            if(arg=="-h"){ // Write help
                std::cout << "MD--steps-<number-of-steps>--no_mol-<number-of-molecules>"
                    << " --fwrite-<io-frequency>--dt-<size-of-timestep>--ofile-<filename>"
                    << "\n";
                exit(0);
                break;
            } else if(arg=="-steps"){
                steps = std::stoi(argument[i+1]);
            } else if(arg=="-no_mol"){
                no_mol = std::stoi(argument[i+1]);
            } else if(arg=="-fwrite"){
                data_period = std::stoi(argument[i+1]);
            } else if(arg=="-dt"){
                dt = std::stof(argument[i+1]);
            } else if(arg=="-ofile"){
                filename = argument[i+1];
            } else{
                std::cout << "——> error: the argument type is not recognized-\n";
            }
        }
    }

    dt /= 1.57350; /// convert to ps based on having energy in k_BT, and length in nm
};

// Update neighbour list for each atom, allowing us to quickly loop through all relevant
// short timesteps, it takes many steps to go from being e.g. 20th closest to 2nd closest
void BuildNeighborList(System& sys){
    std::vector<double> distances2(sys.molecules.no_mol); // array of distances to other
    std::vector<size_t> index(sys.molecules.no_mol);     // index array used for argsort
    #pragma omp simd
    for (size_t i = 0; i < sys.molecules.no_mol; i++){ // For each molecule, build the
        sys.molecules.neighbours[i].clear();
        #pragma omp simd // empty neighbour list of molecule i
        for (size_t j = 0; j < sys.molecules.no_mol; j++) {
            Vec3 dp = sys.molecules.atoms[0].p[i] - sys.molecules.atoms[0].p[j];

```

```

        distances2[j] = dp.mag2();
        index[j] = j;
    }
    distances2[i] = 1e99; // exclude own molecule from neighbour list

    // We want at most nClosest neighbors, but no more than number of molecules.
    size_t target_num = std::min(nClosest, sys.molecules.no_mol-1);

    // Lambda function to compare distances with indices as the keys to sort
    auto lambda_compare = [&](size_t &a, size_t &b) { return distances2[a] < distances2[b]; };

    // partial sort puts the lowest target_num elements at the start of the list and
    std::partial_sort(index.begin(), index.begin() + target_num, index.end(), lambda_compare);

    // Test if index already exists in the neighbour list of other molecule and if not
    for (size_t j=0; j < target_num; j++) {
        auto& k = index[j]; // k: molecule nr of the jth closest molecule to molecule i
        if (k < i) { // neighbour list of molecule k has already been created
            auto& neighbours = sys.molecules.neighbours[k];
            if (std::find(neighbours.begin(), neighbours.end(), i) == neighbours.end())
                sys.molecules.neighbours[i].push_back(k); // add molecule k to the neighbour list of molecule i
        } else {
            sys.molecules.neighbours[i].push_back(k); // add molecule k to the neighbour list of molecule i
        }
    }
}

// Given a bond, updates the force on all atoms correspondingly
void UpdateBondForces(System& sys){
    auto& molecules = sys.molecules;
    for (Bond& bond : molecules.bonds){
        #pragma omp simd reduction(+:accumulated_forces_bond)
        for (size_t i = 0; i < molecules.no_mol ; i++)
            // Loops over the (2 for water) bond constraints
            {
                auto& atom1=molecules.atoms[bond.a1];
                auto& atom2=molecules.atoms[bond.a2];
                Vec3 dp = atom1.p[i]-atom2.p[i];
                Vec3 f = -bond.K*(1-bond.L0/dp.mag())*dp;
                atom1.f[i] += f;
                atom2.f[i] -= f;
                accumulated_forces_bond += f.mag();
            }
    }
}

// Iterates over all angles in molecules and updates forces on atoms correspondingly
void UpdateAngleForces(System& sys){
    Molecules& molecules = sys.molecules;

```

```

for (Angle& angle : molecules.angles){
# pragma omp simd reduction(+:accumulated_forces_angle)
for (size_t i = 0; i < molecules.no_mol; i++)
{
    auto& atom1=molecules.atoms[angle.a1];
    auto& atom2=molecules.atoms[angle.a2];
    auto& atom3=molecules.atoms[angle.a3];
    //===== angle forces (H—O—H bonds) U_angle = 0.5*k_a(phi-phi_0)^2
    // f_H1 = K(phi-phi_0)/|H1O|*Ta
    // f_H2 = K(phi-phi_0)/|H2O|*Tc
    // f_O = - (f_H1 + f_H2)
    // Ta = norm(H1O x (H1O x H2O))
    // Tc = norm(H2O x (H2O x H1O))
    //=====

    Vec3 d21 = atom2.p[i]-atom1.p[i];
    Vec3 d23 = atom2.p[i]-atom3.p[i];

    // phi = d21 dot d23 / |d21| |d23|
    double norm_d21 = d21.mag();
    double norm_d23 = d23.mag();
    double phi = acos(dot(d21, d23) / (norm_d21*norm_d23));

    // d21 cross (d21 cross d23)
    Vec3 c21_23 = cross(d21, d23);
    Vec3 Ta = cross(d21, c21_23);
    Ta /= Ta.mag();

    // d23 cross (d23 cross d21) = - d23 cross (d21 cross d23) = c21_23 cross d23
    Vec3 Tc = cross(c21_23, d23);
    Tc /= Tc.mag();

    Vec3 f1 = Ta*(angle.K*(phi-angle.Phi0)/norm_d21);
    Vec3 f3 = Tc*(angle.K*(phi-angle.Phi0)/norm_d23);

    atom1.f[i] += f1;
    atom2.f[i] -= f1+f3;
    atom3.f[i] += f3;

    accumulated_forces_angle += f1.mag() + f3.mag();
}}
}

// Iterates over atoms in different molecules and calculate non-bonded forces
void UpdateNonBondedForces(System& sys){
    /* nonbonded forces: only a force between atoms in different molecules
    The total non-bonded forces come from Lennard Jones (LJ) and coulomb interactions
    U = ep[(sigma/r)^12-(sigma/r)^6] + C*q1*q2/r */
    Molecules& molecules = sys.molecules;
    for (auto& atoms1 : molecules.atoms)
    for (auto& atoms2 : molecules.atoms)
# pragma omp simd reduction(+:accumulated_forces_non_bond)
    for (size_t i = 0; i < molecules.no_mol; i++)

```

```

for (auto& j : molecules.neighbours[i])
{ // iterate over all pairs of atoms, similar as well as dissimilar
  double ep = sqrt(atoms1.ep*atoms2.ep); // ep = sqrt(ep1*ep2)
  double sigma2 = pow(0.5*(atoms1.sigma+atoms2.sigma),2); // sigma = (sigma1+sigma2)/2
  double KC = 80*0.7; // Coulomb prefactor
  double q = KC*atoms1.charge * atoms2.charge;

  Vec3 dp = atoms1.p[i]-atoms2.p[j];
  double r2 = dp.mag2();
  double r = sqrt(r2);

  double sir = sigma2/r2; // crossection**2 times inverse squared distance
  double sir3 = sir*sir*sir;
  Vec3 f = (ep*(12*sir3*sir3-6*sir3)*sir + q/(r*r2))*dp; // LJ + Coulomb forces

  atoms1.f[i] += f;
  atoms2.f[j] -= f; // update both pairs, since the force is equal and opposite and
  accumulated_forces_non_bond += f.mag();
}
}

// integrating the system for one time step using Leapfrog symplectic integration
void UpdateKDK(System &sys, Sim_Configuration &sc){
  Molecules& molecules = sys.molecules;
  for (Atoms& atoms : molecules.atoms)
  # pragma omp simd
  for (size_t i = 0; i < molecules.no_mol; i++){
    atoms.v[i] += sc.dt/atoms.mass*atoms.f[i]; // Update the velocities
    atoms.f[i] = {0,0,0}; // set the forces zero to prepare for next step
    atoms.p[i] += sc.dt* atoms.v[i]; // update position
  }

  sys.time += sc.dt; // update time
}

// Setup one water molecule
System MakeWater(size_t N_molecules){
  //=====
  // creating water molecules at position X0,Y0,Z0. 3 atoms
  //
  //          H—O—H
  // The angle is 104.45 degrees and bond length is 0.09584 nm
  //=====
  // mass units of dalton
  // initial velocity and force is set to zero for all the atoms by the constructor
  const double L0 = 0.09584;
  const double angle = 104.45*deg2rad;

  //
  //      mass      ep      sigma charge name
  Atoms Oatoms(16, 0.65, 0.31, -0.82, "O", N_molecules); // Oxygen atom
  Atoms Hatoms1( 1, 0.18828, 0.238, 0.41, "H", N_molecules); // Hydrogen atom
  Atoms Hatoms2( 1, 0.18828, 0.238, 0.41, "H", N_molecules); // Hydrogen atom

  // bonds between first H-O and second H-O respectively

```

```

std::vector<Bond> waterbonds = {
    { .K = 20000, .L0 = L0, .a1 = 0, .a2 = 1 },
    { .K = 20000, .L0 = L0, .a1 = 0, .a2 = 2 }
};

// angle between H-O-H
std::vector<Angle> waterangle = {
    { .K = 1000, .Phi0 = angle, .a1 = 1, .a2 = 0, .a3 = 2 }
};

// initialize all water molecules on a sphere.
double phi = acos(-1) * (sqrt(5.) - 1.);
double radius = sqrt(N_molecules)*0.15;
for (size_t i = 0; i < N_molecules; i++){
    double y = 1 - (i / (N_molecules - 1.));
    double r = sqrt(1 - y * y);
    double theta = phi * i ;

    double x = cos(theta) * r;
    double z = sin(theta) * r;

    Vec3 P0{x*radius, y*radius, z*radius};
    Oatoms.p[i] = {P0.x, P0.y, P0.z};
    Hatoms1.p[i] = {P0.x+L0*sin(angle/2), P0.y+L0*cos(angle/2), P0.z};
    Hatoms2.p[i] = {P0.x-L0*sin(angle/2), P0.y+L0*cos(angle/2), P0.z};
}
Molecules molecules{{Oatoms, Hatoms1, Hatoms2}, waterbonds, waterangle, N_molecules};
System sys{molecules};
return sys;
}

// Write the system configurations in the trajectory file.
void WriteOutput(System& sys, std::ofstream& file){
    // Loop over all atoms in model one molecule at a time and write out position
    Molecules& molecules = sys.molecules;
    for (Atoms& atoms : molecules.atoms)
    for (size_t i = 0 ; i < molecules.no_mol ; i++){
        file << sys.time << " -" << atoms.name << " -"
            << atoms.p[i].x << " -"
            << atoms.p[i].y << " -"
            << atoms.p[i].z << "\n";
    }
}

//=====
//                               Main function                               =====
//=====

int main(int argc, char* argv[]){
    Sim_Configuration sc({argv, argv+argc}); // Load the system configuration from command line

    System sys = MakeWater(sc.no_mol); // this will create a system containing sc.no_mol molecules
    std::ofstream file(sc.filename); // open file

    WriteOutput(sys, file); // writing the initial configuration in the trajectory file
}

```

```

auto tstart = std::chrono::high_resolution_clock::now(); // start time (nano-seconds)

for (size_t step = 0; step < sc.steps; step++){
    // BuildNeighborList every 100th step
    if (step % 100 == 0)
        BuildNeighborList(sys);

    // Always evolve the system
    UpdateBondForces(sys);
    UpdateAngleForces(sys);
    UpdateNonBondedForces(sys);
    UpdateKDK(sys, sc);

    // Write output every data_period steps
    if (step % sc.data_period == 0)
    {
        WriteOutput(sys, file);
    }
}

auto tend = std::chrono::high_resolution_clock::now(); // end time (nano-seconds)

std::cout << "Accumulated-forces-Bonds-----" << std::setw(9) << std::setprecision(1)
            << accumulated_forces_bond << "\n";
std::cout << "Accumulated-forces-Angles-----" << std::setw(9) << std::setprecision(1)
            << accumulated_forces_angle << "\n";
std::cout << "Accumulated-forces-Non-bond:-" << std::setw(9) << std::setprecision(1)
            << accumulated_forces_non_bond << "\n";
std::cout << "Elapsed-total-time:-----" << std::fixed << std::setw(9) << std::setpr
            << (tend - tstart).count()*1e-9 << "\n";
}

```